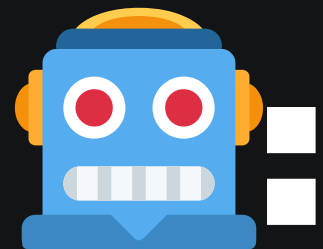


上下文工程

记忆



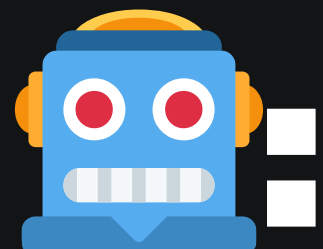
“我是小明”



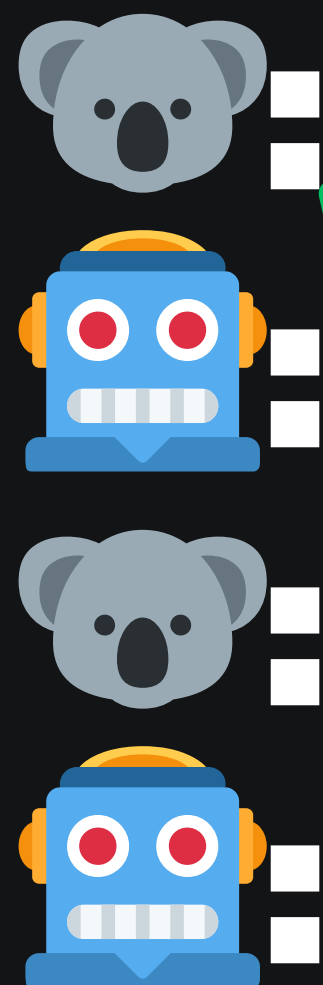
“你好小明!”



“我叫什么?”



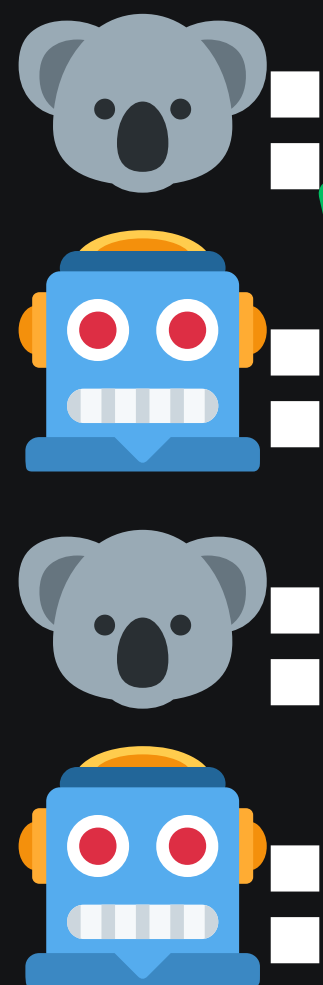
“我不清楚你叫什么”



“我是**小明**”

“你好小明!”

“我叫什么?”



“我是小明”

“你好小明!”

“我叫什么?”

“你叫小明”

(口播)

我把所有的对话记忆都给到大模型不就行了？



# **3个限制:**

**1. 模型上下文窗口的限制**

**2. 费Token!**

**3. 过多的上下文可导致输出质量持平甚至下降**

# 记忆管理策略

| 记忆管理策略 | 原理          | 优点 | 缺点 | 应用场景 |
|--------|-------------|----|----|------|
| 窗口记忆   | 只记住最近的N论对话  |    |    |      |
| 总结记忆   | 将较早的对话压缩成摘要 |    |    |      |
| 结构化记忆  | 提取保存重要的事实信息 |    |    |      |

# 记忆管理策略

| 记忆管理策略 | 原理          | 优点         | 缺点     | 应用场景                   |
|--------|-------------|------------|--------|------------------------|
| 窗口记忆   | 只记住最近的N论对话  | 简单，开销小，响应快 | 长期记忆丧失 | 通用聊天机器人，客服<br>查询订单产品规格 |
| 总结记忆   | 将较早的对话压缩成摘要 |            |        |                        |
| 结构化记忆  | 提取保存重要的事实信息 |            |        |                        |



# 记忆管理策略

| 记忆管理策略 | 原理          | 优点           | 缺点                 | 应用场景                     |
|--------|-------------|--------------|--------------------|--------------------------|
| 窗口记忆   | 只记住最近的N论对话  | 简单，开销小，响应快   | 长期记忆丧失             | 通用聊天机器人，客服<br>查询订单产品规格   |
| 总结记忆   | 将较早的对话压缩成摘要 | 延长记忆跨度，上下文连贯 | 信息压缩损失，额外开销，效果依赖模型 | 长内容/会议创作分析，个性化学习与辅导，复杂项目 |
| 结构化记忆  | 提取保存重要的事实信息 |              |                    |                          |

# 记忆管理策略

| 记忆管理策略 | 原理          | 优点             | 缺点                 | 应用场景                       |
|--------|-------------|----------------|--------------------|----------------------------|
| 窗口记忆   | 只记住最近的N论对话  | 简单，开销小，响应快     | 长期记忆丧失             | 通用聊天机器人，客服<br>查询订单产品规格     |
| 总结记忆   | 将较早的对话压缩成摘要 | 延长记忆跨度，上下文连贯   | 信息压缩损失，额外开销，效果依赖模型 | 长内容/会议创作分析，个性化学习与辅导，复杂项目   |
| 结构化记忆  | 提取保存重要的事实信息 | 记忆精确可靠，可解释与可编辑 | 复杂度高(提取，检索，存储)，延迟  | 企业知识库问答，法律、医疗或金融等领域，个人智能助理 |

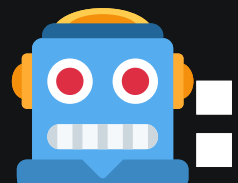
# 记忆管理策略

| 记忆管理策略 | 原理          | 优点             | 缺点                 | 应用场景                       |
|--------|-------------|----------------|--------------------|----------------------------|
| 窗口记忆   | 只记住最近的N论对话  | 简单，开销小，响应快     | 长期记忆丧失             | 通用聊天机器人，客服<br>查询订单产品规格     |
| 总结记忆   | 将较早的对话压缩成摘要 | 延长记忆跨度，上下文连贯   | 信息压缩损失，额外开销，效果依赖模型 | 长内容/会议创作分析，个性化学习与辅导，复杂项目   |
| 结构化记忆  | 提取保存重要的事实信息 | 记忆精确可靠，可解释与可编辑 | 复杂度高(提取，检索，存储)，延迟  | 企业知识库问答，法律、医疗或金融等领域，个人智能助理 |

# 滑动窗口记忆 $k=2$



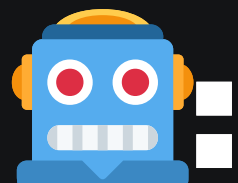
“第一个密码是 “月光” ”



“好的，我记住了。”



“第二个密码是 “河流” 。”

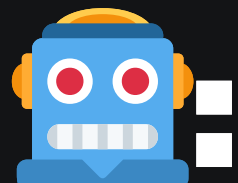


“好的，我记住了。”

# 滑动窗口记忆 $k=2$



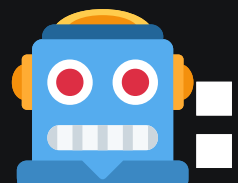
“第一个密码是“月光””



“好的，我记住了。”



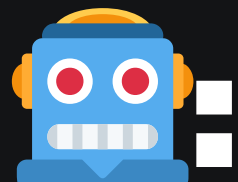
“第二个密码是“河流”。”



“好的，我记住了。”

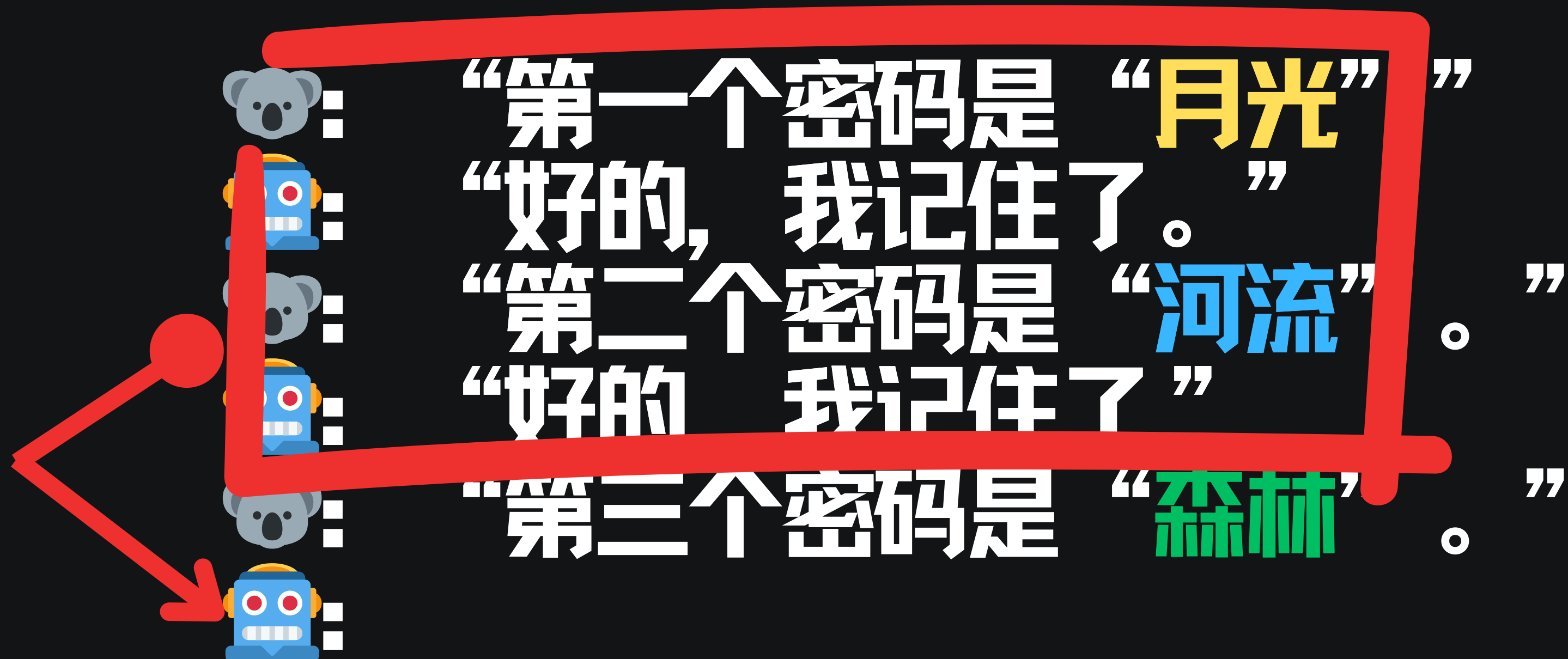


“第三个密码是“森林”。”





# 滑动窗口记忆 $k=2$



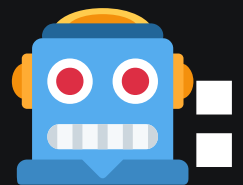
# 滑动窗口记忆 $k=2$



# 滑动窗口记忆 $k=2$



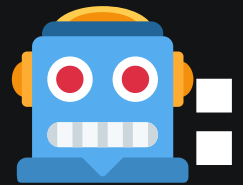
“第一个密码是“月光””



“好的，我记住了。”



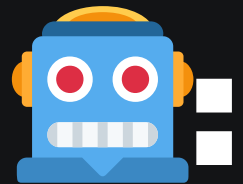
“第二个密码是“河流”。



“好的，我记住了。”



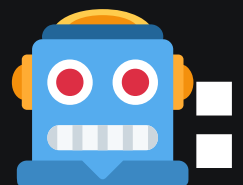
“第三个密码是“森林”。



“好的，我记住了。”



“第一个密码是什么？”





# 滑动窗口记忆 $k=2$



“第一个密码是“月光””



“好的，我记住了”



“第二个密码是“河流””



“好的，我记住了。”



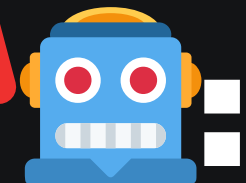
“第三个密码是“森林””



“好的，我记住了”



“第一个密码是什么？”



# 滑动窗口记忆 $k=2$



“第一个密码是“月光””



“好的，我记住了。”



“第二个密码是“河流””



“好的，我记住了。”



“第三个密码是“森林””



“好的，我记住了。”



“第一个密码是什么？”



“我不知道。”



# 讲代码的目的不是

一行一行写出代码

# 而是

会读代码，了解基本的编程和AI Agent框架知识，从而：

1. 精确的向AI提要求和需求+技术选型
2. 验证AI产出的代码(目前AI公司不能背锅)
3. 更好的与AI交互，一定程度上掌控自己的项目
4. 更好的融入自己的领域知识

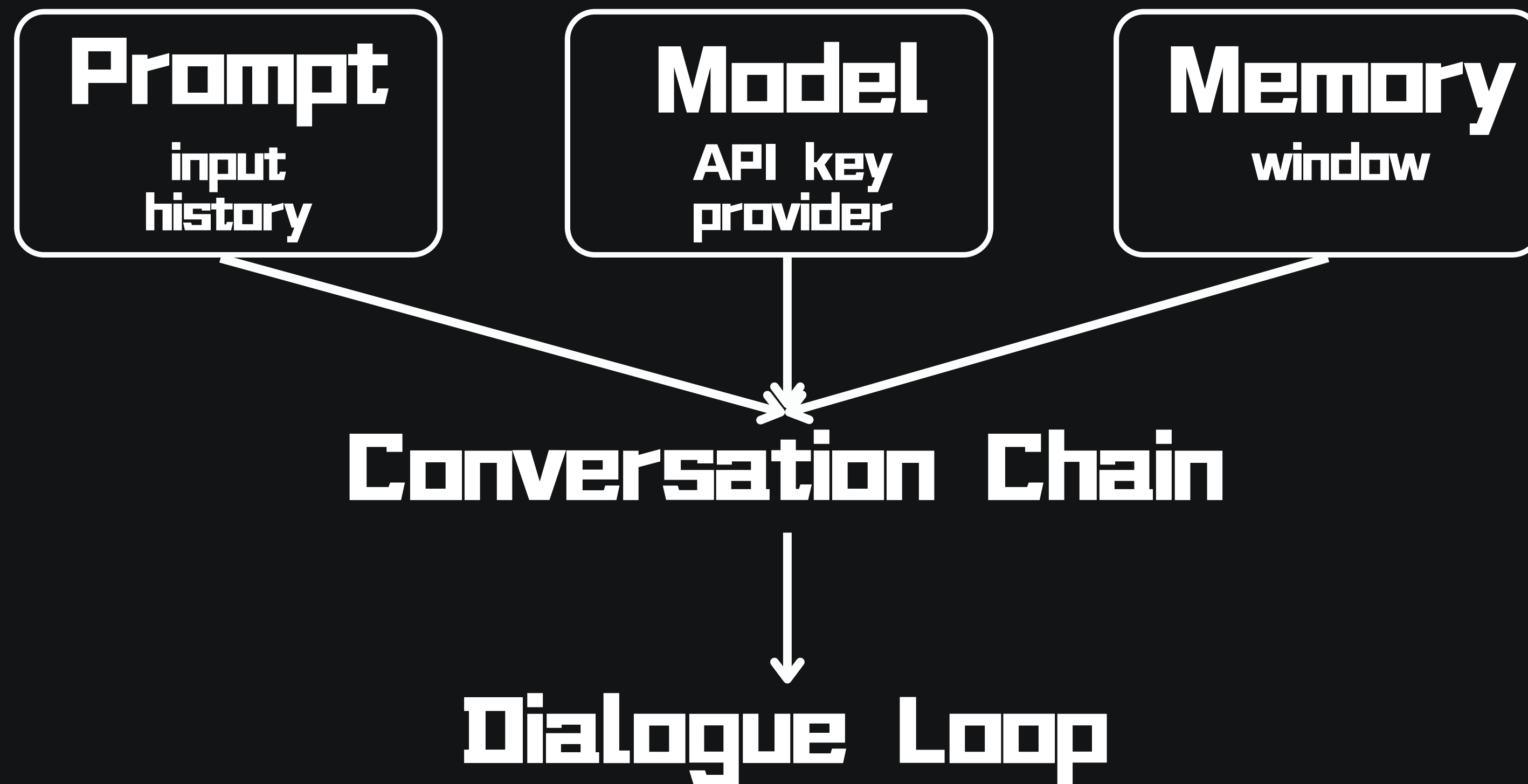
即使不亲自砌每一块砖，也有必要懂建筑图纸和工程原理。

# 代码1结构[零编程基础]

旧版Langchain

# 代码1结构[零编程基础]

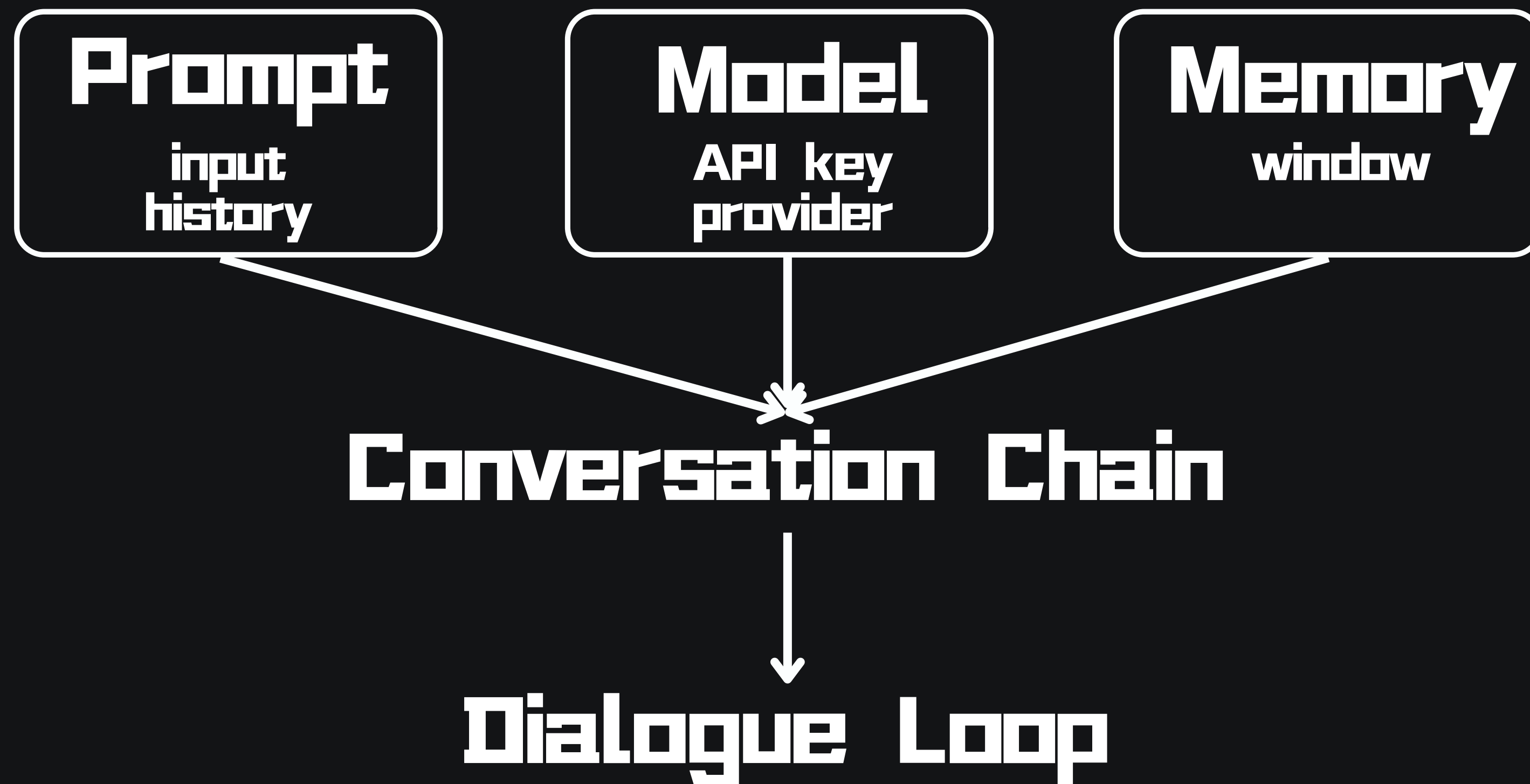
旧版langchain





# 代码1结构[零编程基础]

旧版Langchain



LangChain 的本质就是“连接”。它将一个孤立的 LLM (大脑), 与外部的记忆、数据和工具连接起来, 并通过“链”这个行动框架进行统一调度, 从而构建出强大、实用的AI应用。

# 代码1结构[零编程基础] 旧版Langchain

**准备工作:**  
下载VScode,  
安装uv,  
创建虚拟环境,  
初始化项目

**Github:**

**<https://github.com/TianyiDataScience/Context-Engineering-Tutorial>**



# 代码1结构[零编程基础] 旧版Langchain

## 0. 下载VS code:

a. <https://code.visualstudio.com/>

## 1. 安装uv:

a. Linux/MacOS:

```
curl -Lsf https://astral.sh/uv/install.sh | sh
```

后重启终端

b. Windows:

```
irm https://astral.sh/uv/install.ps1 | iex
```

后重启powershell

Markdown ▾ Copy Caption ...

?什么是uv?

uv是用于 Python 的一个非常快速的包安装器和解析器

uv 主要能做到两件事情，并将它们做得极快:

1. 包管理: 作为 pip 和 pip-tools 的一个完整且极速的替代品。
2. 虚拟环境管理: 作为 venv 和 virtualenv 的一个快速替代品。

## 2. 设置项目并运行 AI 代理

```
mkdir my-ai-agent
cd my-ai-agent
```

## 3. 创建并激活虚拟环境: uv 会自动在名为 .venv 的文件夹中创建环境。

a. Linux/MacOS:

```
# 创建虚拟环境
uv venv
# 激活虚拟环境
source .venv/bin/activate
```

b. Windows

```
# 创建虚拟环境
uv venv
# 激活虚拟环境
.\.venv\Scripts\Activate.ps
```

## 4. 初始化项目: 此命令会创建一个 pyproject.toml 文件。

```
uv init
```

## 5. 安装依赖库:

```
uv add langchain langchain_community python-dotenv
```

## 6. 创建 Python 脚本文件:

```
# linux/mac
touch chat_agent.py
# windows
ni chat_agent.py
```

## 7. 在项目中创建 .env 文件

```
# 确保你在 my-ai-agent 文件夹内
# linux/mac
touch .env
# windows
ni .env
```

打开 .env 文件，然后像下面这样写入您的密钥（注意：没有 export，格式是 KEY=VALUE）:

```
GOOGLE_API_KEY='你的密钥'
```

## 8. 确保向 Python 程序传递中文字符时，使用的编码格式是 Python 和 LangChain 所期望的 UTF-8。

```
# WSL/Linux
echo 'export PYTHONUTF8=1' >> ~/.bashrc
```

然后启动新的终端

## 9. 粘贴代码: 将您的 AI 代理代码粘贴到打开的编辑器中，保存。

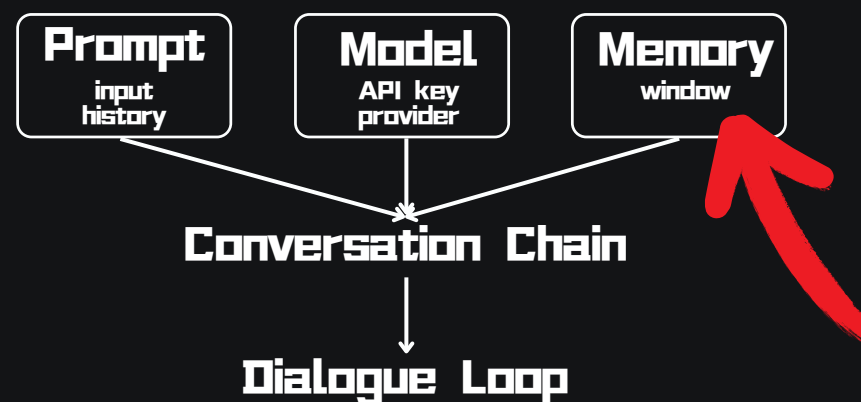
## 10. 运行脚本:

```
python chat_agent.py
```



# 代码1结构[零编程基础]

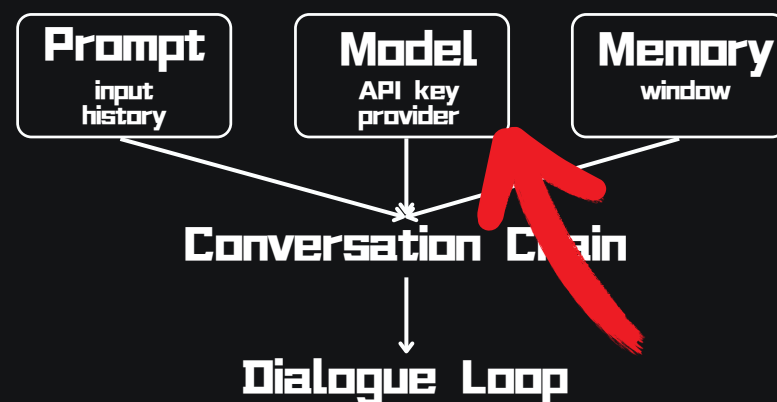
旧版Langchain



```
memory = ConversationBufferWindowMemory(k=2)
```

# 代码1结构[零编程基础]

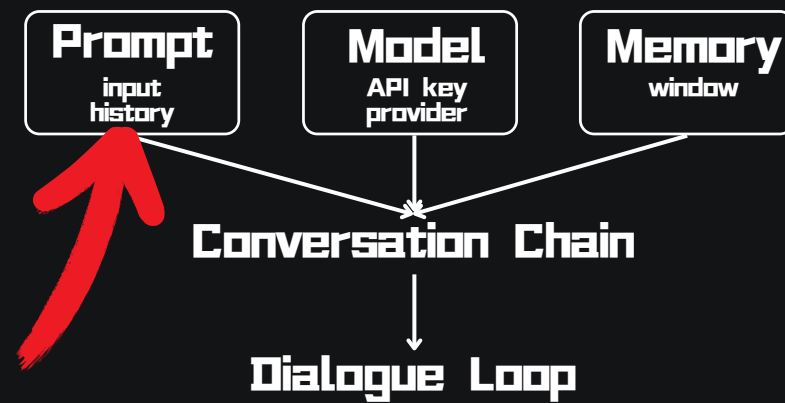
旧版Langchain



```
llm= init_chat_model(model="gemini-2.5-flash",model_provider="google_genai")
```

# 代码1结构[零编程基础]

旧版Langchain



```
template = """
请你回答用户的问题，但不要总结对话历史

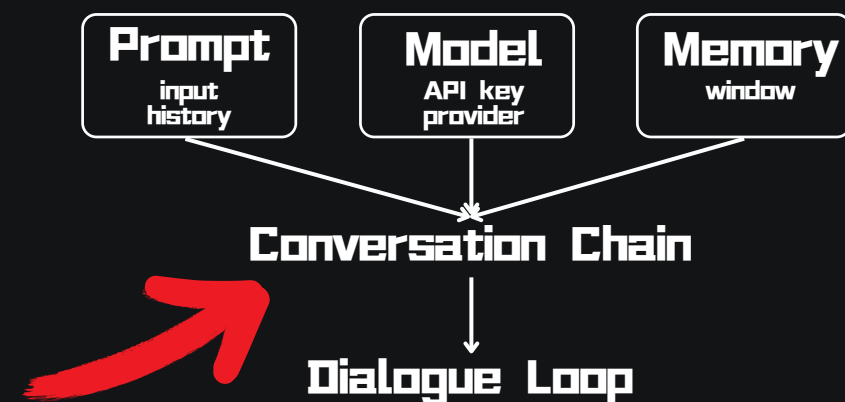
当前的对话历史：
{history}

用户: {input}
AI:
"""

prompt = PromptTemplate(
    input_variables=["history", "input"],
    template=template
)
```

# 代码1结构[零编程基础]

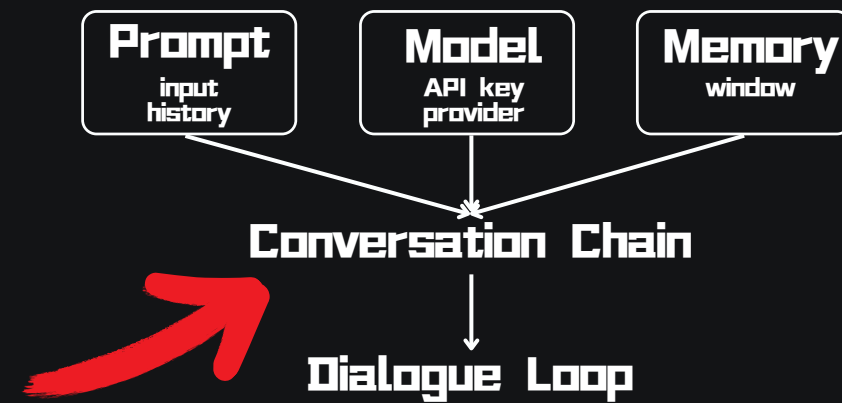
旧版Langchain



```
30 # --- 3. 创建一个封装好的、带滑动窗口的记忆对象 ---
31 memory = ConversationBufferWindowMemory(k=2) # k=2 表示记住最近2轮对话
```

# 代码1结构[零编程基础]

旧版Langchain

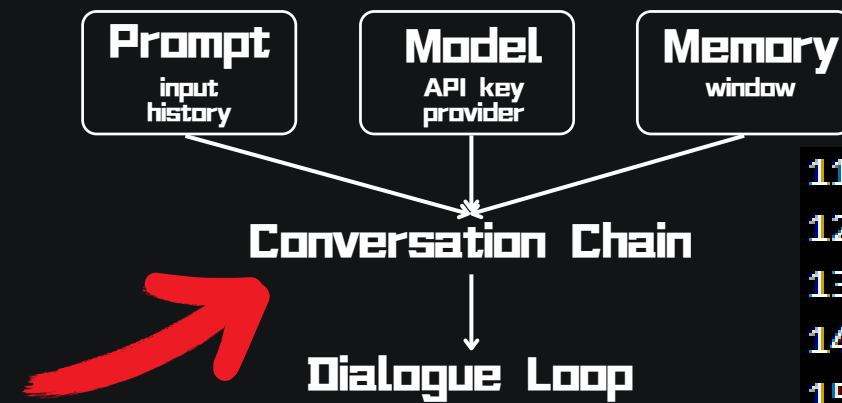


```
27 # --- 2. 初始化模型 ---
28 llm= init_chat_model(model="gemini-2.5-flash",model_provider="google_genai")
29
30 # --- 3. 创建一个封装好的、带滑动窗口的记忆对象 ---
31 memory = ConversationBufferWindowMemory(k=2) # k=2 表示记住最近2轮对话
```



# 代码1结构[零编程基础]

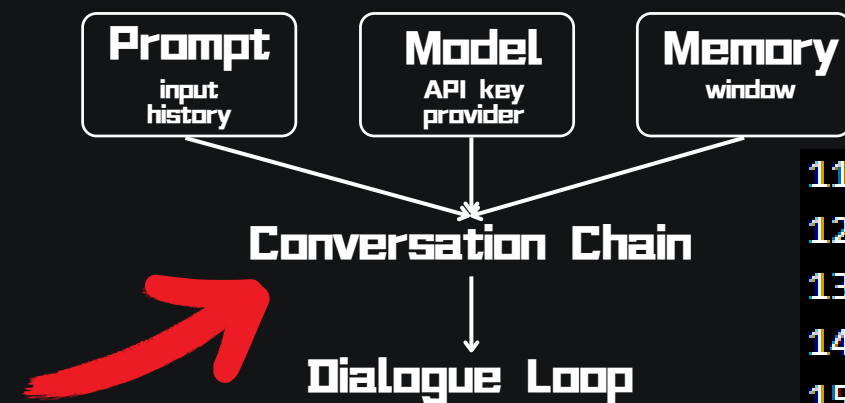
旧版Langchain



```
11 # --- 1. 定义提示词模板 ---
12 template = """
13 请你回答用户的问题，但不要总结对话历史
14
15 当前的对话历史：
16 {history}
17
18 用户：{input}
19 AI：
20 """
21
22 prompt = PromptTemplate(
23     input_variables=["history", "input"],
24     template=template
25 )
26
27 # --- 2. 初始化模型 ---
28 llm= init_chat_model(model="gemini-2.5-flash",model_provider="google_genai")
29
30 # --- 3. 创建一个封装好的、带滑动窗口的记忆对象 ---
31 memory = ConversationBufferWindowMemory(k=2) # k=2 表示记住最近2轮对话
```

# 代码1结构[零编程基础]

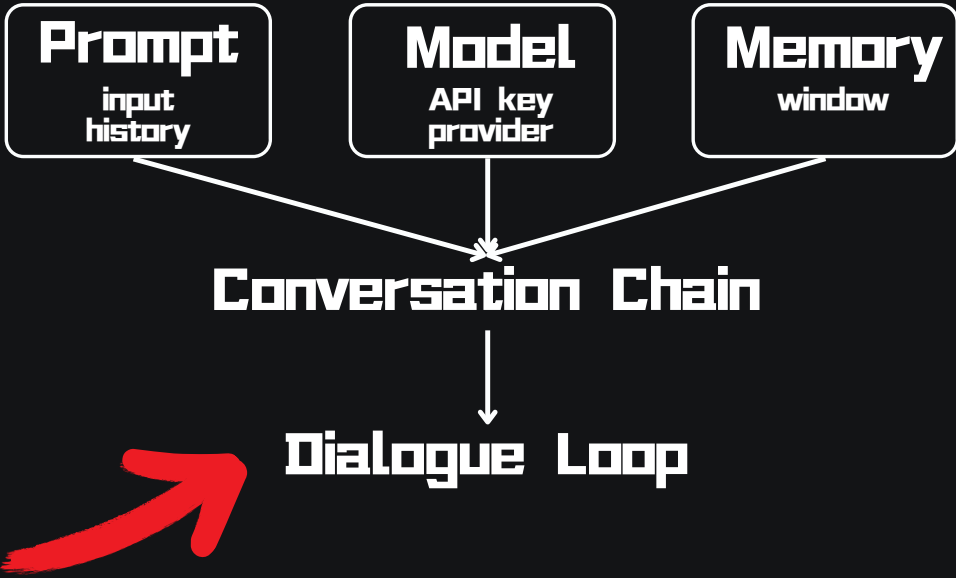
旧版Langchain



```
11 # --- 1. 定义提示词模板 ---
12 template = """
13 请你回答用户的问题，但不要总结对话历史
14
15 当前的对话历史：
16 {history}
17
18 用户：{input}
19 AI：
20 """
21
22 prompt = PromptTemplate(
23     input_variables=["history", "input"],
24     template=template
25 )
26
27 # --- 2. 初始化模型 ---
28 llm= init_chat_model(model="gemini-2.5-flash",model_provider="google_genai")
29
30 # --- 3. 创建一个封装好的、带滑动窗口的记忆对象 ---
31 memory = ConversationBufferWindowMemory(k=2) # k=2 表示记住最近2轮对话
32
33 # --- 4. 创建一个封装好的对话链 ---
34 conversation = ConversationChain(
35     llm=llm,
36     memory=memory,
37     prompt=prompt,
38     verbose=True
39 )
```

# 代码1结构[零编程基础]

旧版Langchain



代码演示



**旧版langchain高度封装，简单易学，为什么还需要新版呢？**

1. 旧版本: 黑箱。整个过程( 记忆读取 -> Prompt填充 -> LLM调用) 被封装得严严实实。缺少灵活和自由。
2. 新版本: 自定义能力, 支持流式输出 (Streaming), 方并行处理, 与langsmith集成

# 代码2结构[一点编程基础]

新版Langchain

# 代码2结构[一点编程基础]

新版Langchain

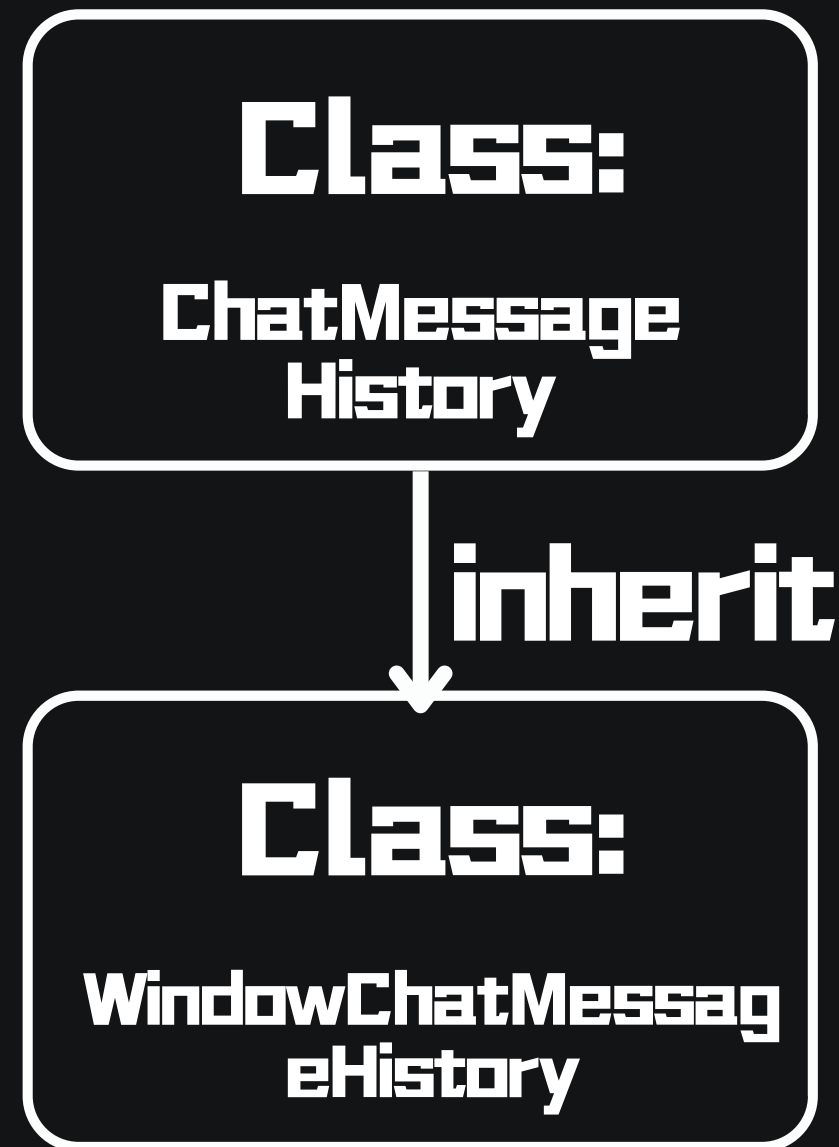
**Class:**

**ChatMessage  
History**

在内存中( RAM) 以存储和管理对话消息  
默认会一直存储添加的所有消息, 没有任何大小限制

# 代码2结构[一点编程基础]

新版langchain



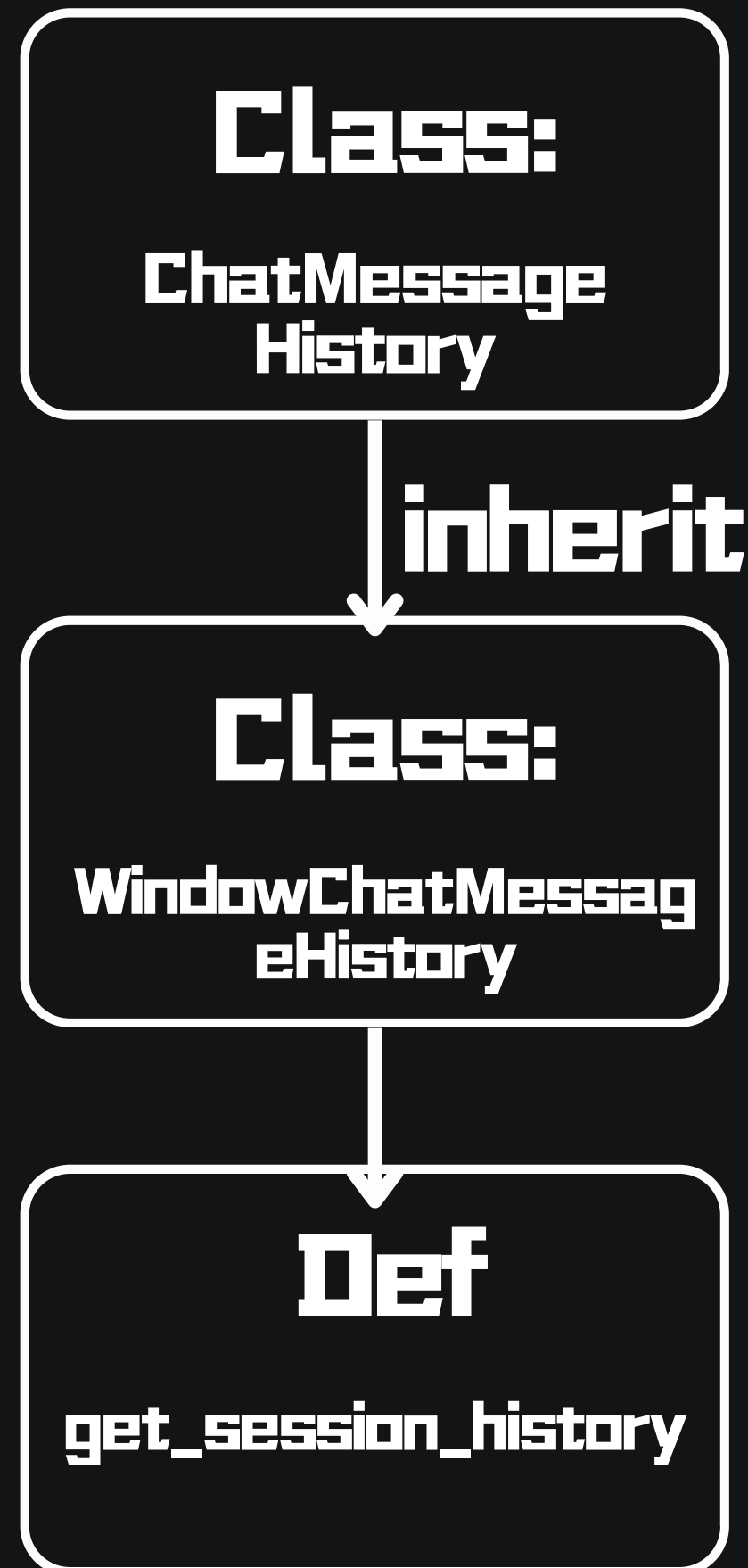
在内存中( RAM) 以存储和管理对话消息  
默认会一直存储添加的所有消息, 没有任何大小限制

实现滑动窗口效果



# 代码2结构[一点编程基础]

新版langchain



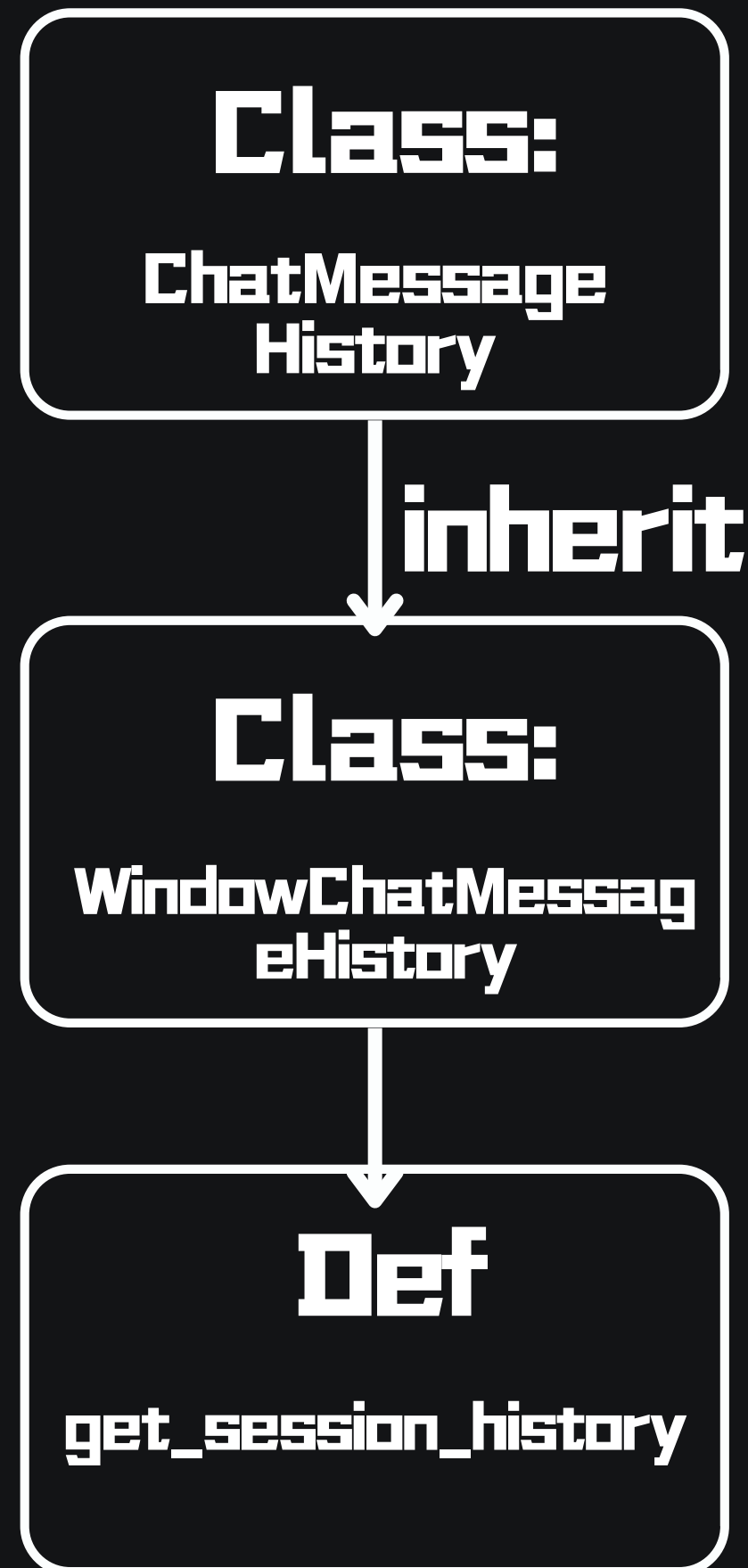
在内存中( RAM) 以存储和管理对话消息  
默认会一直存储添加的所有消息, 没有任何大小限制

实现滑动窗口效果

为每个会话ID创建一个独立的、带窗口的记忆实例

# 代码2结构[一点编程基础]

新版langchain



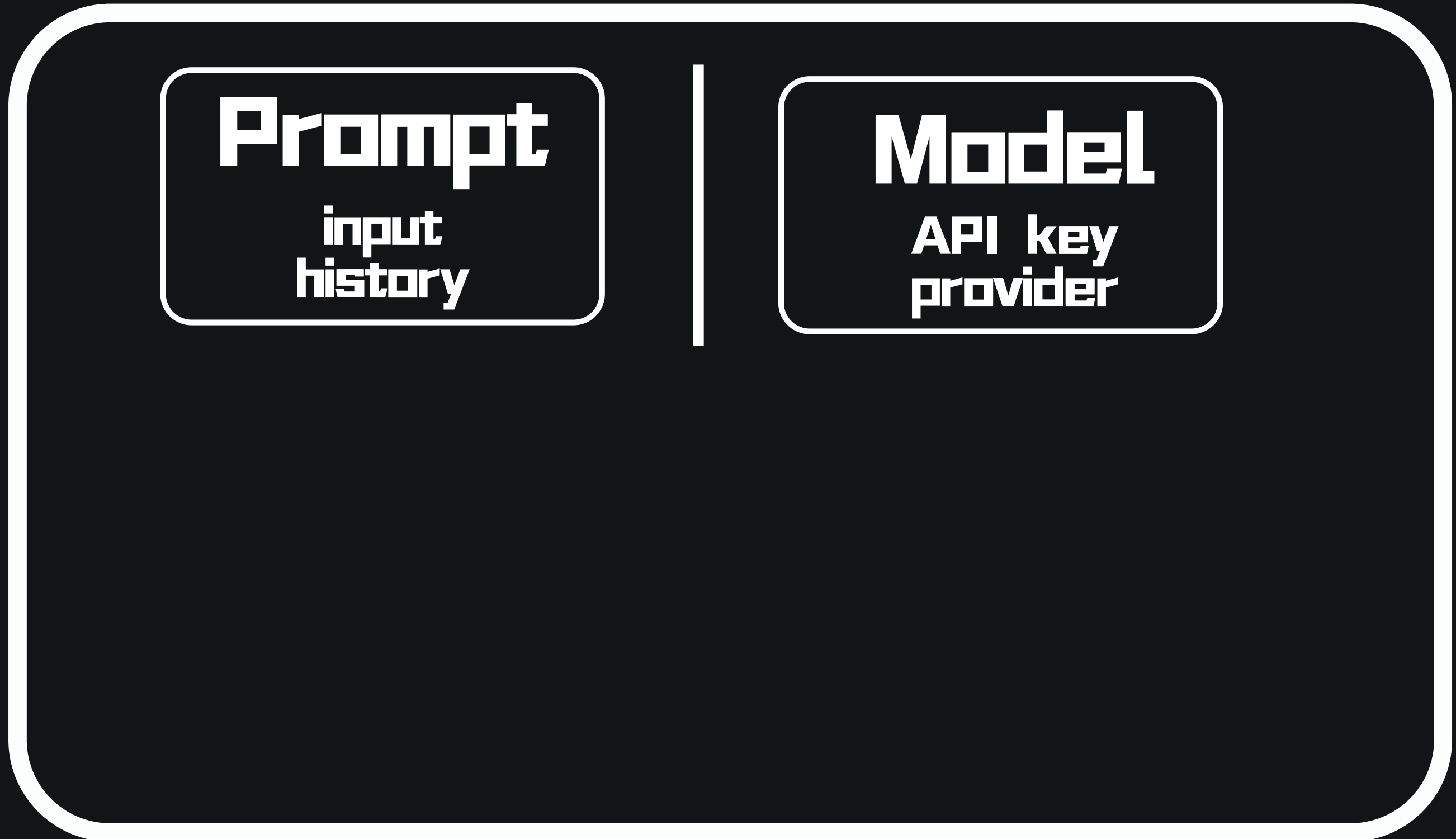
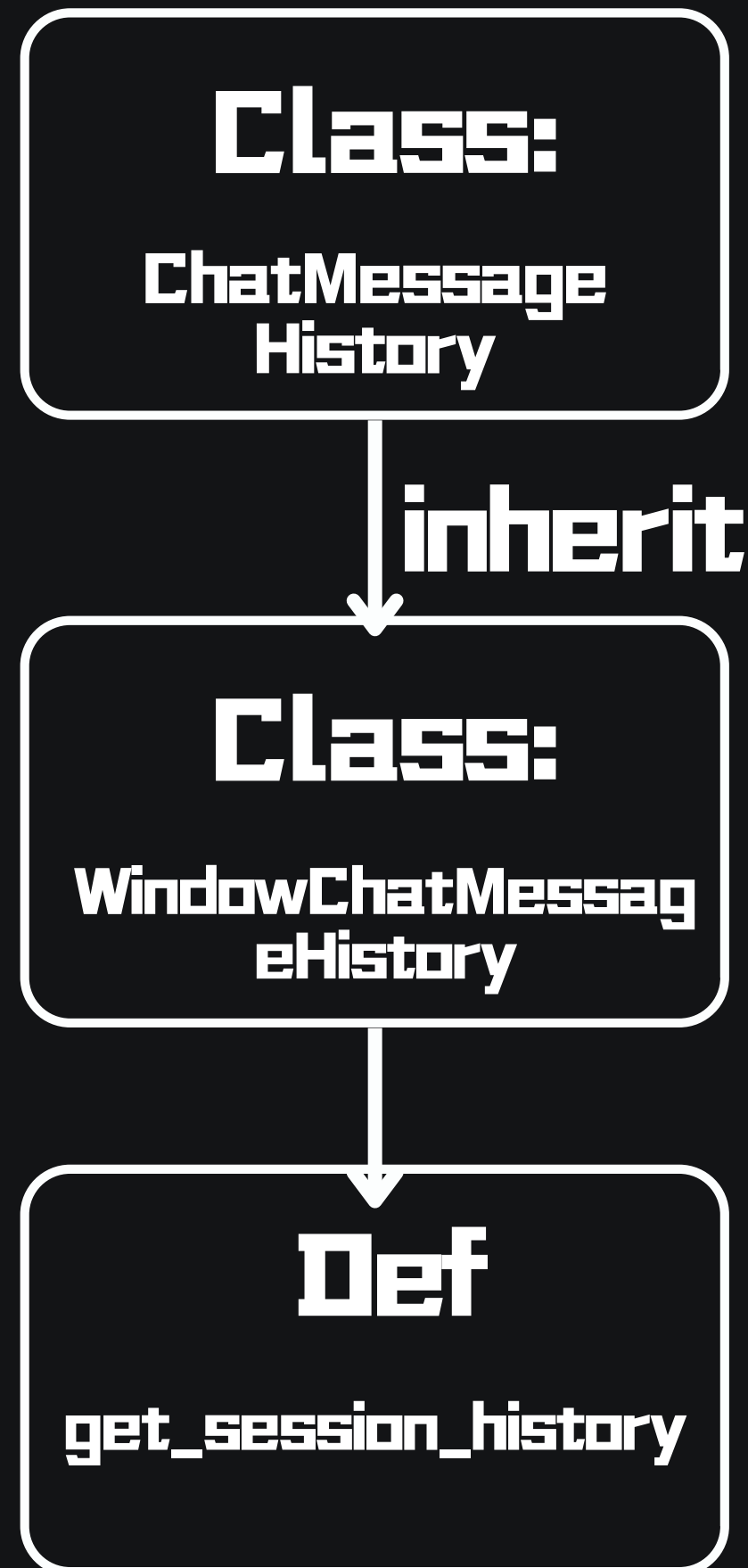
在内存中( RAM) 以存储和管理对话消息  
默认会一直存储添加的所有消息, 没有任何大小限制

实现滑动窗口效果

为每个会话ID创建一个独立的、带窗口的记忆实例

# 代码2结构[一点编程基础]

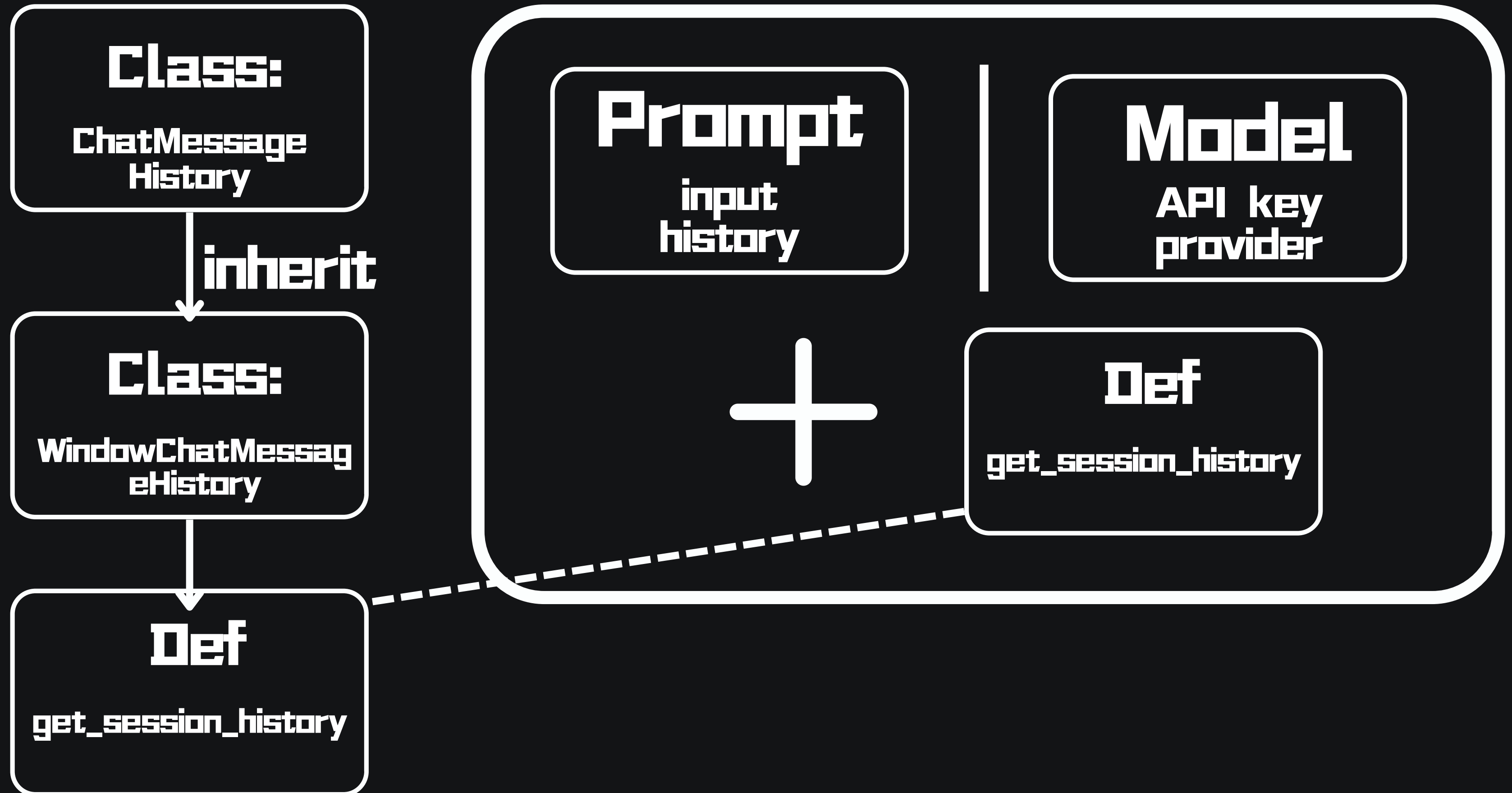
新版Langchain





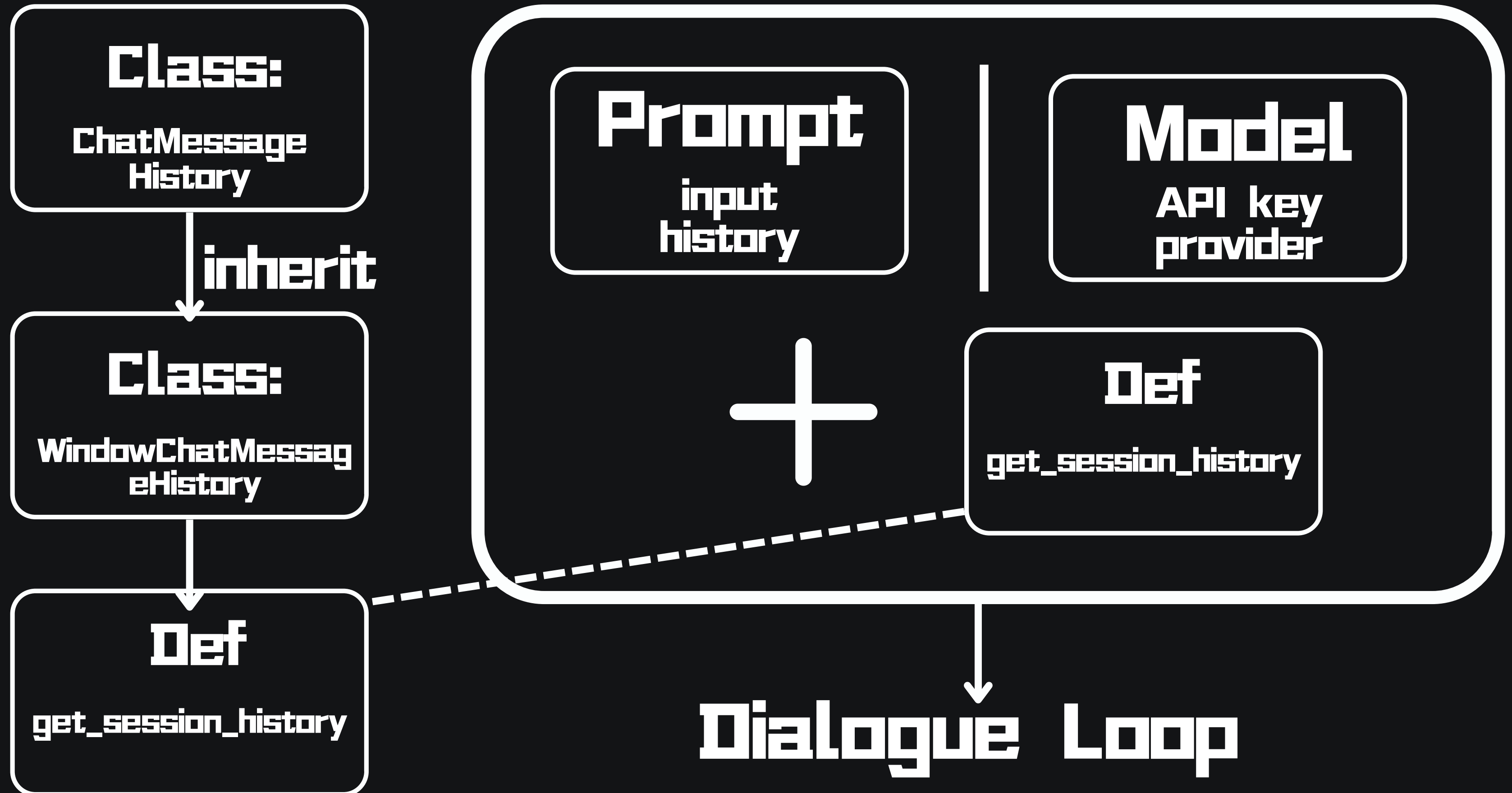
# 代码2结构[一点编程基础]

新版Langchain



# 代码2结构[一点编程基础]

新版Langchain



# 代码2结构[一点编程基础]

新版Langchain

**Class:**

**ChatMessage  
History**

inherit

**Class:**

**WindowChatMessag  
eHistory**

```
9  # --- 1. 自定义带窗口功能的 ChatMessageHistory 子类 ---
10 class WindowChatMessageHistory(ChatMessageHistory):
11     """一个自动修剪消息以符合指定窗口大小的 ChatMessageHistory。"""
12
13     k: int #对话轮数
14     window_size: int #窗口大小
15
16     # 调用父类的 __init__ 构造函数: 为新创建的对象设置其初始状态 (属性)
17     def __init__(self, k: int):
18         super().__init__(k=k, window_size=k * 2)
19
20     # 通过继承 ChatMessageHistory 并重写 (override) add_message 方法。
21     def add_message(self, message: BaseMessage) -> None:
22         """添加一条消息, 并在需要时修剪旧消息"""
23         super().add_message(message)
24
25         if len(self.messages) > self.window_size:
26             self.messages.pop(0)
27
```

# 代码2结构[一点编程基础]

新版Langchain

**Class:**

**ChatMessage  
History**

**inherit**

**Class:**

**WindowChatMessag  
eHistory**

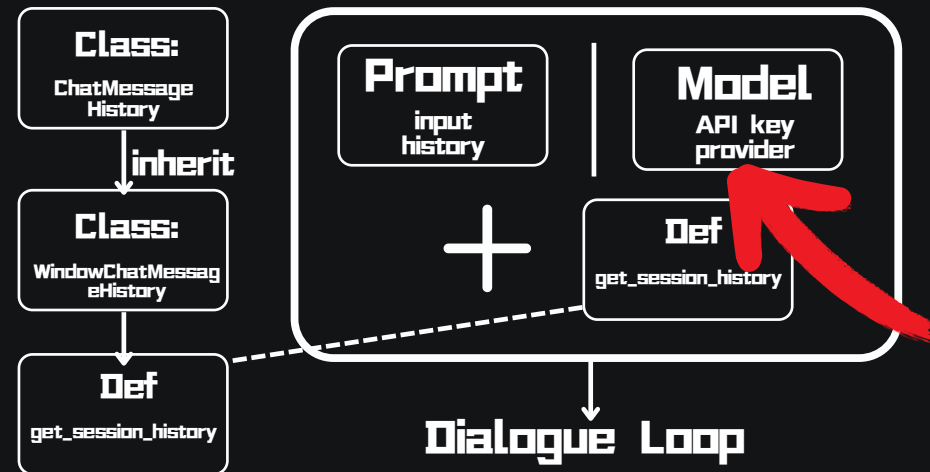
**Def**

**get\_session\_history**

```
9  # --- 1. 自定义带窗口功能的 ChatMessageHistory 子类 ---
10 class WindowChatMessageHistory(ChatMessageHistory):
11     """一个自动修剪消息以符合指定窗口大小的 ChatMessageHistory。"""
12
13     k: int #对话轮数
14     window_size: int #窗口大小
15
16     # 调用父类的 __init__ 构造函数: 为新创建的对象设置其初始状态 (属性)
17     def __init__(self, k: int):
18         super().__init__(k=k, window_size=k * 2)
19
20     # 通过继承 ChatMessageHistory 并重写 (override) add_message 方法。
21     def add_message(self, message: BaseMessage) -> None:
22         """添加一条消息, 并在需要时修剪旧消息"""
23         super().add_message(message)
24
25         if len(self.messages) > self.window_size:
26             self.messages.pop(0)
27
28 # --- 2. 每个会话ID创建一个独立的、带窗口的记忆实例 ---
29 store = {}
30 def get_session_history(session_id: str) -> WindowChatMessageHistory:
31     if session_id not in store:
32         # k=2 表示只记住最近2轮对话
33         store[session_id] = WindowChatMessageHistory(k=2)
34     return store[session_id]
```

# 代码2结构[一点编程基础]

新版Langchain

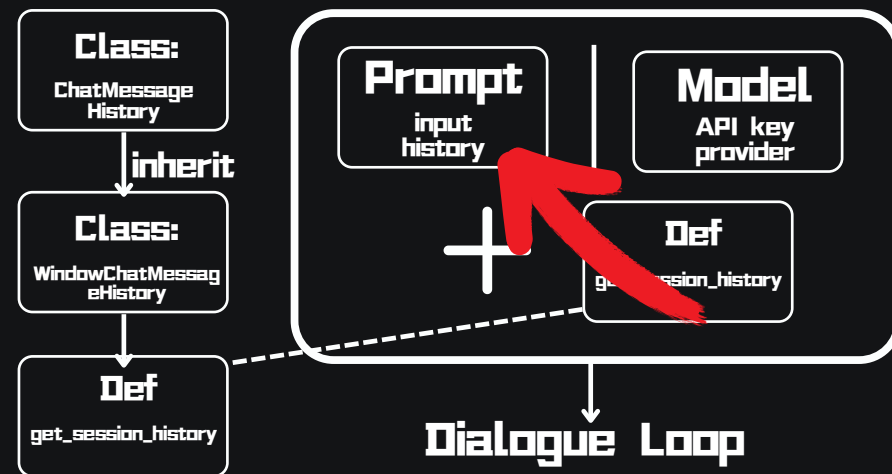


```
42 # 初始化LLM
43 model = init_chat_model(model="gemini-2.5-flash",model_provider="google_genai")
```



# 代码2结构[一点编程基础]

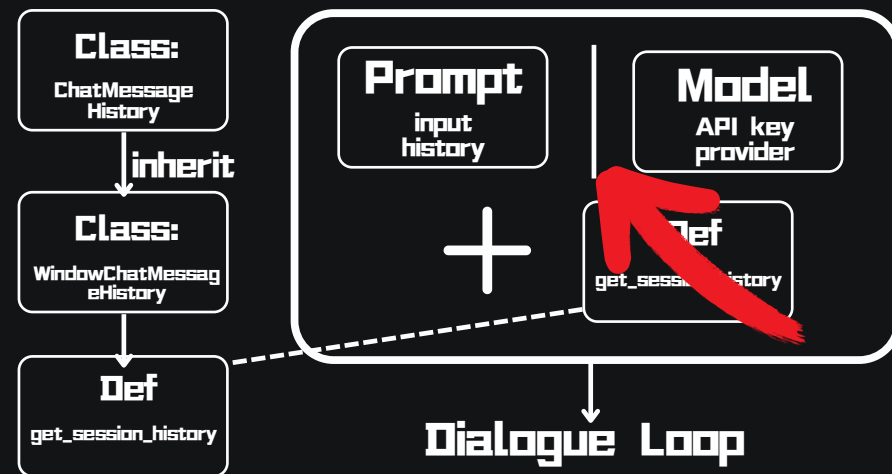
新版Langchain



```
42 # 初始化LLM
43 model = init_chat_model(model="gemini-2.5-flash",model_provider="google_genai")
44
45 # 定义提示模板，history部分会自动由RunnableWithMessageHistory填充
46 prompt = ChatPromptTemplate.from_messages([
47     ("system", "请你回答用户的问题，但不要总结对话历史。"),
48     MessagesPlaceholder(variable_name="history"),
49     ("human", "{input}"),
50 ])
```

# 代码2结构[一点编程基础]

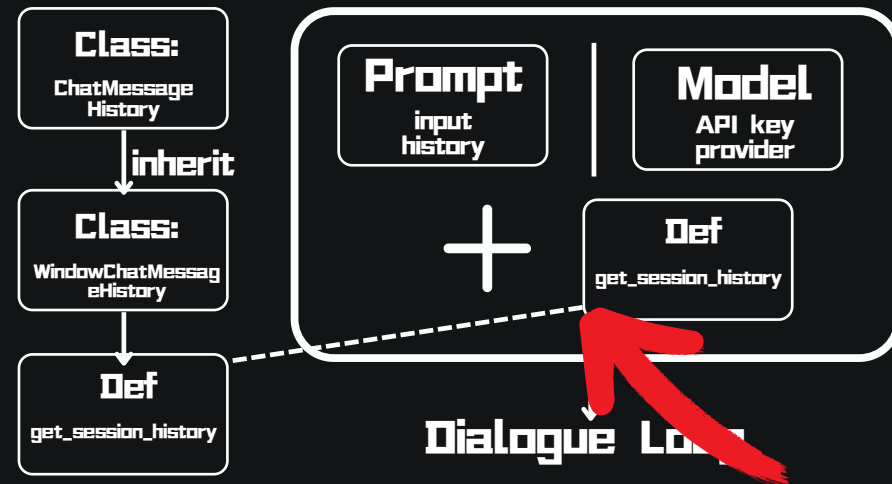
新版Langchain



```
42 # 初始化LLM
43 model = init_chat_model(model="gemini-2.5-flash",model_provider="google_genai")
44
45 # 定义提示模板，history部分会自动由RunnableWithMessageHistory填充
46 prompt = ChatPromptTemplate.from_messages([
47     ("system", "请你回答用户的问题，但不要总结对话历史。"),
48     MessagesPlaceholder(variable_name="history"),
49     ("human", "{input}"),
50 ])
51
52 # 创建核心的对话链
53 runnable = prompt | model
```

# 代码2结构[一点编程基础]

新版Langchain

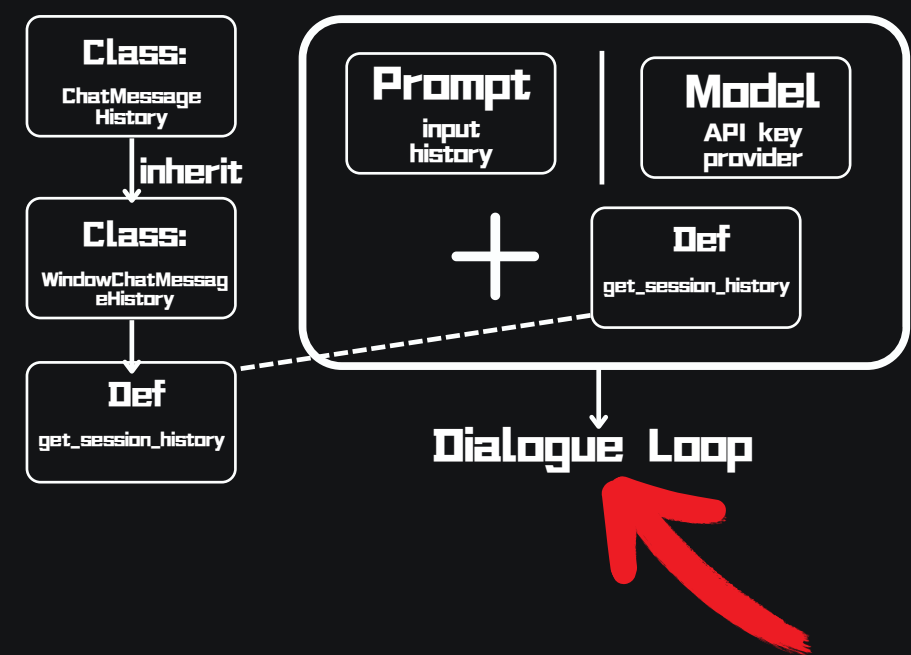


```
42 # 初始化LLM
43 model = init_chat_model(model="gemini-2.5-flash",model_provider="google_genai")
44
45 # 定义提示模板，history部分会自动由RunnableWithMessageHistory填充
46 prompt = ChatPromptTemplate.from_messages([
47     ("system", "请你回答用户的问题，但不要总结对话历史。"),
48     MessagesPlaceholder(variable_name="history"),
49     ("human", "{input}"),
50 ])
51
52 # 创建核心的对话链
53 runnable = prompt | model
54
55 # 使用RunnableWithMessageHistory包装核心链，为其添加记忆功能
56 chat_agent = RunnableWithMessageHistory(
57     runnable,
58     get_session_history,
59     input_messages_key="input",
60     history_messages_key="history",
61 )
```



# 代码2结构[一点编程基础]

新版Langchain



代码演示